Sistemas Operacionais II

Projeto III
Propagação Imediata de Mensagens

Felipe Murta, Felipe Gassen, Gustavo Dangelo, Tom Hunt

Objetivos

- Corrigir erros de arquitetura das entregas I e II.
- Implementar interrupções assíncronas para propagação instântena de mensagens.
- Implementar um fluxo de recebimento zero-copy em pacotes recebidos.

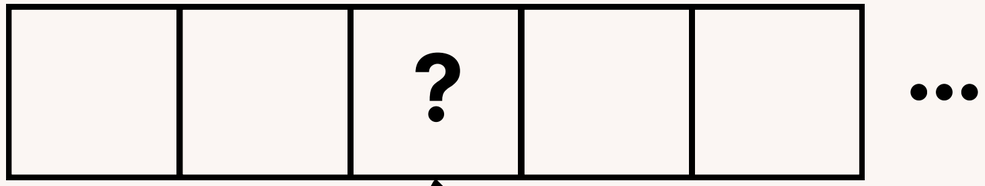
Correção de erros anteriores

- Uso de Spin-Waits
- Inicialização da Memória Compartilhada com arquivo

Remoção de Spin-Waits

Antes

Buffer

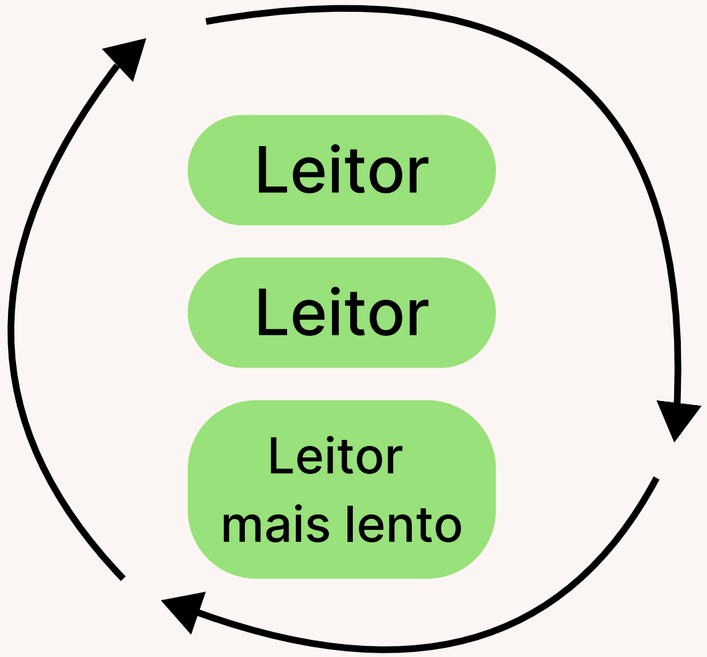


Escritor não sabe se o slot do buffer dele já está disponível para escrita

Escritor

while (true)

for() { }



break OU usleep(100)

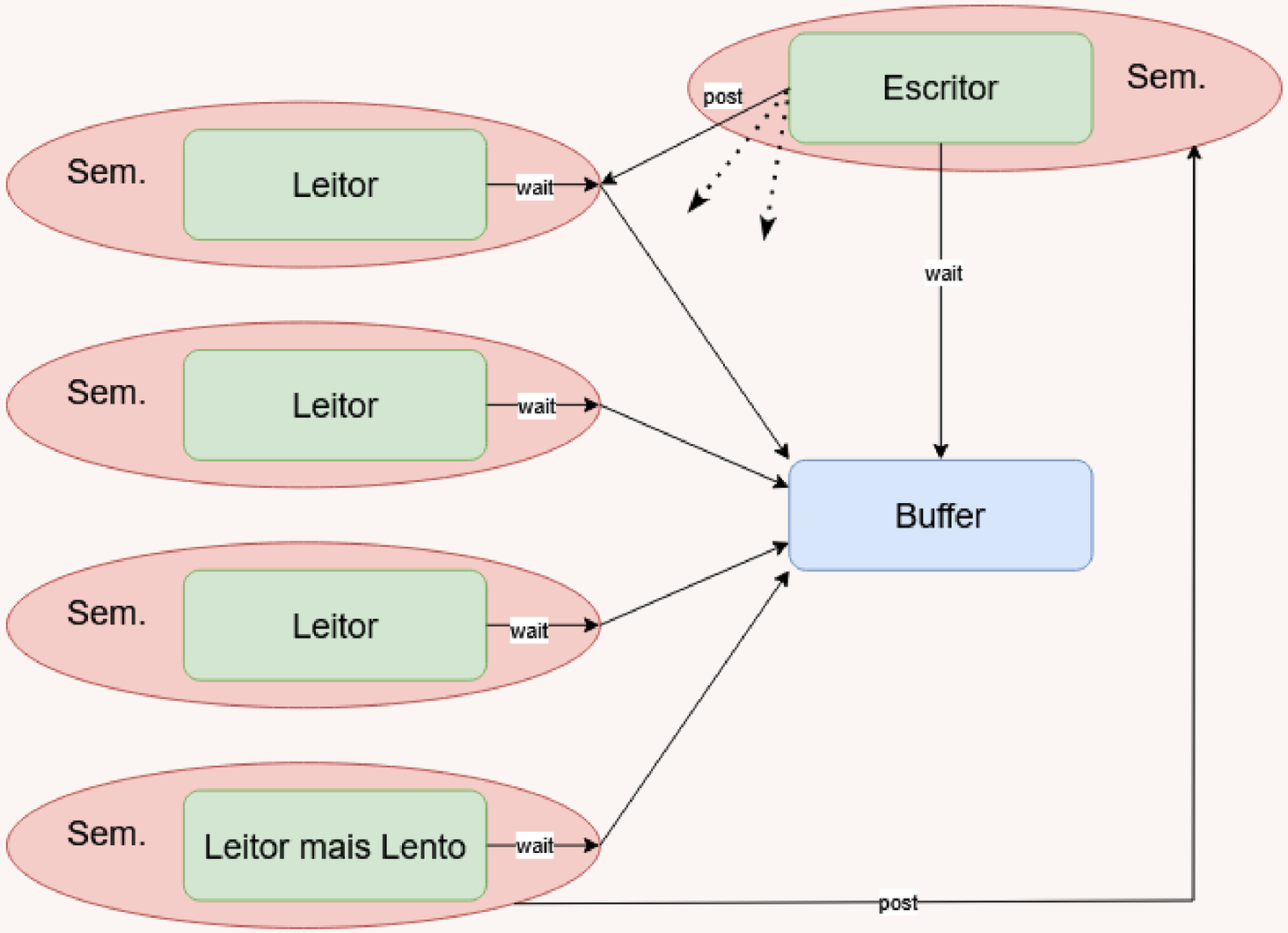
Remoção de Spin-Waits

Antes

```
while (true) {
            uint64_t slowest_reader_id = my_ticket_id; // Assume no readers are behind
            bool readers_active = false;
            for (int i = 0; i < MAX_CLIENTS; ++i) {
                if (_shared_block->client_registry[i].is_active) {
                    readers_active = true;
                    uint64_t last_read = _shared_block->client_registry[i].last_read_sequence_id;
                    slowest_reader_id = std::min(slowest_reader_id, last_read);
                }
            }

            if (!readers_active || (my_ticket_id - slowest_reader_id < BUFFER_SLOTS)) {
                break; // Slot is free to be written
            }
            std::cout << "[WARNING] Writer stuck on slow readers" << std::endl;
            usleep(100); // Wait for slow readers to catch up
        }
```

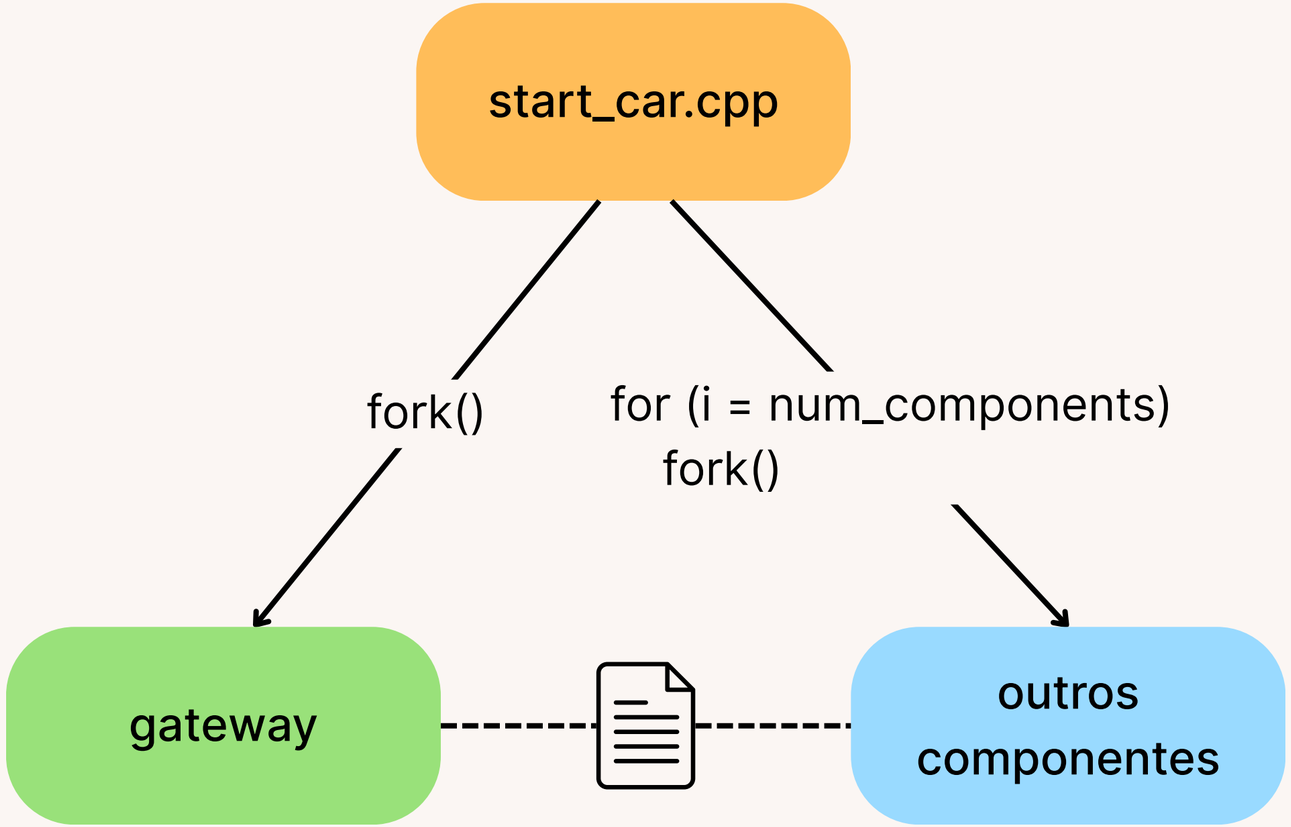
Remoção de Spin-Waits

Solução



Inicialização da Shared Memory

Antes



- Um dos componentes cria um arquivo, chama ftok(), e incia a **shm** e **set de semaforos.**
- Próximos componentes dão attach na **shm** e dão get no **set de semáforos** pelo arquivo já criado.

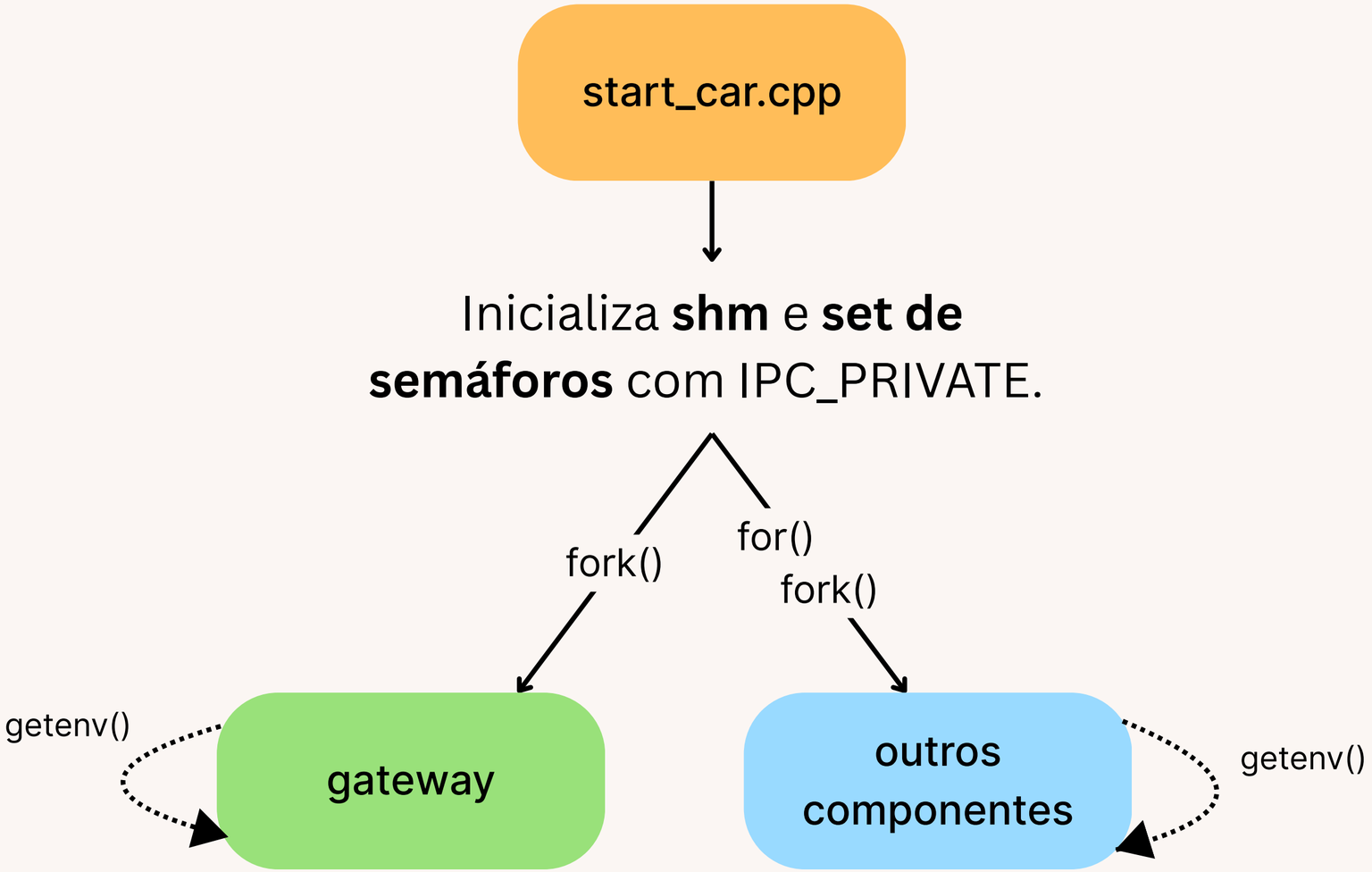
Inicialização da Shared Memory

Antes

```
ShmEngine() {

        FILE* fp = fopen(SHM_KEY_FILE, "a"); // creating/touching the file if it doesn't exist (required by ftok)
        if (fp) {
            fclose(fp);
        } else {
            _cleanupAndThrow("Failed to create IPC key file");
        }

        // generating the key for shared memory access. See [1]
        key_t shared_key = ftok(SHM_KEY_FILE, PROJ_ID);
        if (shared_key == -1) {
            _cleanupAndThrow("Failed to generate IPC key");
        }

        // allocating or getting the shared memory segment. See [2]
        _shared_id = shmget(shared_key, sizeof(SharedBlock), 0666 | IPC_CREAT);
        if (_shared_id == -1) {
            _cleanupAndThrow("Failed to create/get shared memory segment");
        }
```

Inicialização da Shared Memory

Solução



- Key IPC_PRIVATE
- **Nova restrição:** memória compartilhada entre processos pais e filhos.
- **Solução:** start_car.cpp inicia as estruturas compartilhadas ANTES dos forks(), e salva os IDs em variáveis de ambiente.

Inicialização da Shared Memory

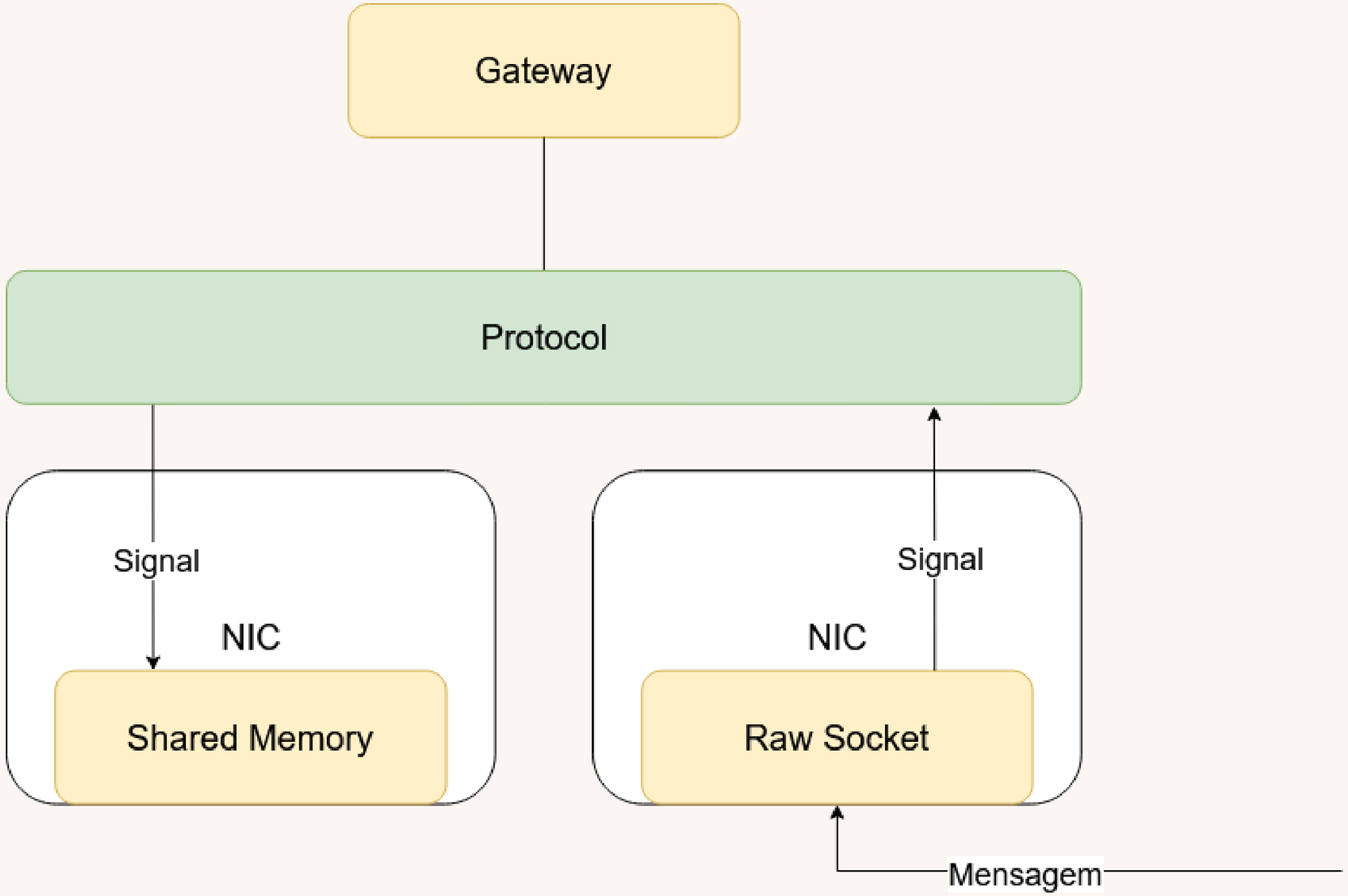
Solução

ShmEngine:

```
static bool bootstrapIPC() {
        // 1. Creates shared memory using IPC_PRIVATE. Only creates
        int shared_id = -1;
        shared_id = shmget(IPC_PRIVATE, sizeof(SharedBlock), 0666 | IPC_CREAT | IPC_EXCL);
        if (shared_id == -1) {
            perror("IPC Bootstrap: failed to create/get shared memory segment");
            return false;
        }

        // 2. Attaches the shared memory segment to the parent process (running start_car).
        // This will allow creating the shared block, which requires a shm pointer
        void* shared_pointer = shmat(shared_id, nullptr, 0);
        if (shared_pointer == (void *)-1) {
            perror("IPC Bootstrap: failed to attach memory segment");
            shmctl(shared_id, IPC_RMID, nullptr); // marks the shared memory segment to be destroyed. See [7].
            return false;
        }
```

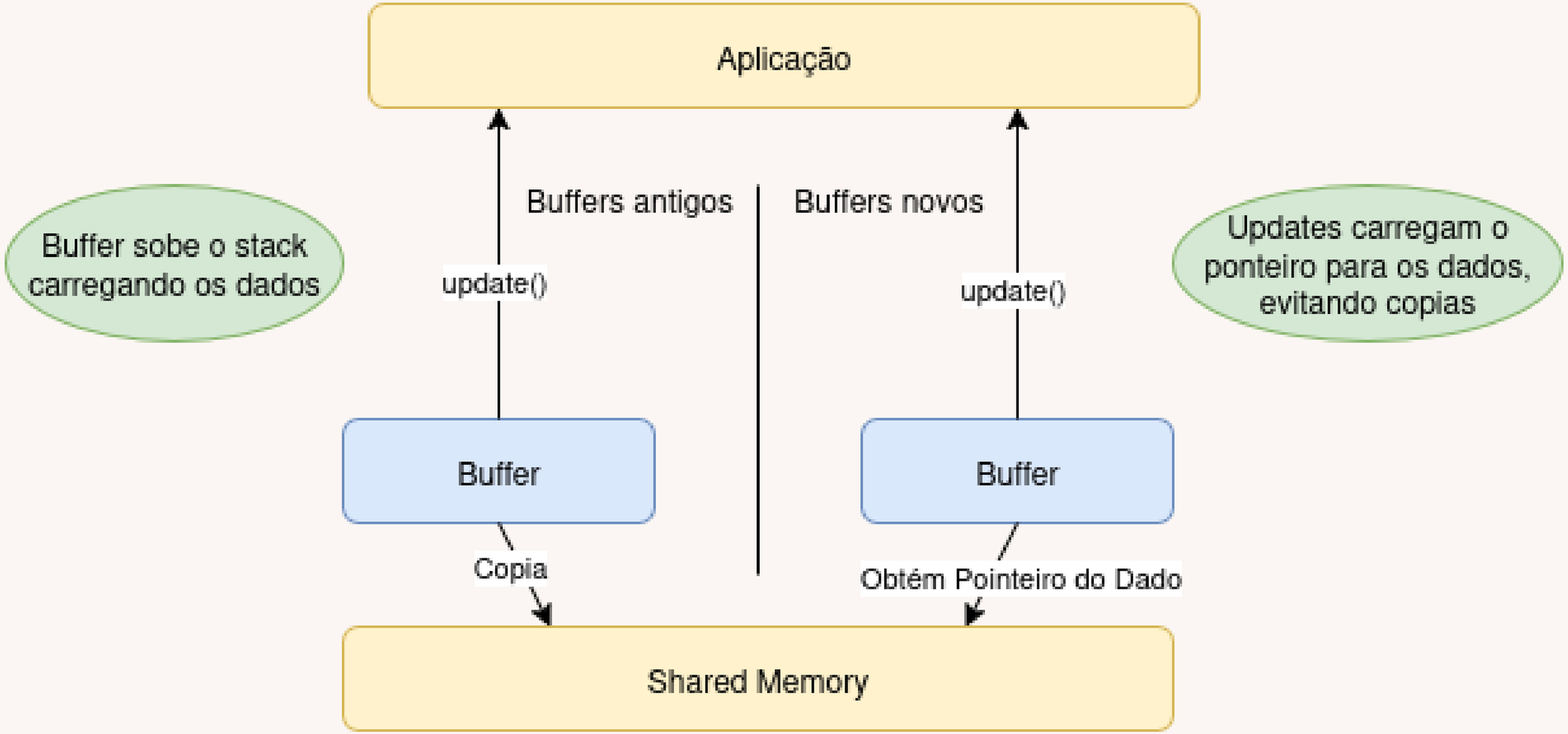
Propagação Imediata de Mensagens

Propagação Imediata de Mensagens



Buffers Zero-Copy

Buffers Zero-Copy



Resultados e Conclusões

- Latência
- Objetivos Futuros

Testes de Latência

RTT: Round-Trip Time

(entre componentes de veículos distintos)

Latência Média:
1.1971 ms

Latência Máxima:
2.7194 ms

Latência Mínima:
0.3060 ms

```
--- Statistics for logs/vm1.log ---
Samples: 104
Min: 0.4506 ms
Max: 2.1563 ms
Average: 1.2323 ms
Std Dev: 0.3824 ms

--- Statistics for logs/vm2.log ---
Samples: 100
Min: 0.3060 ms
Max: 2.2441 ms
Average: 1.1599 ms
Std Dev: 0.4479 ms

--- Statistics for logs/vm3.log ---
Samples: 100
Min: 0.4327 ms
Max: 2.0022 ms
Average: 1.2404 ms
Std Dev: 0.3528 ms

--- Statistics for logs/vm4.log ---
Samples: 108
Min: 0.4026 ms
Max: 1.9341 ms
Average: 1.1468 ms
Std Dev: 0.3118 ms

--- Statistics for logs/vm5.log ---
Samples: 104
Min: 0.3807 ms
Max: 2.7194 ms
Average: 1.2494 ms
Std Dev: 0.4969 ms

=== Overall Statistics (all VMs) ===
Total samples: 516
```

Futuros Objetivos

- Aperfeiçoar a tipagem de mensagens e componentes para permitir modelos Publish-Subscribe (Objetivo do P4).
- Realizar a Sincronização de Relógios (Objetivo do P5).
- Refatorar a classe component para melhor legibilidade.